

Spring Security от первого фильтра до JWT, refresh и OAuth 2.0

Подробный учебный конспект для проекта `warehouse-spring-demo`

Версия контекста: август 2026. Java 17, Spring Boot 3.5.5, Spring Security 6, JJWT 0.13.0, PostgreSQL, Liquibase.

Как читать: сначала понять модель «завода», затем сверяться со схемами, потом смотреть алгоритм и код. Кодовые примеры привязаны к текущему репозиторию; refresh-обмен и logout на момент создания документа ещё не завершены.

Оглавление

- | | |
|---|--------------------------------------|
| 1. Зачем веб-приложению безопасность | 7. JWT: теория и реализация |
| 2. Словарь основных понятий | 8. JWT + refresh token |
| 3. HTTP, сервлеты, фильтры и Spring Security «из коробки» | 9. OAuth 2.0 |
| 4. Виды аутентификации и 2FA | 10. OpenID Connect |
| 5. Базовая настройка Spring Security | 11. Как реализовать OAuth 2.0 / OIDC |
| 6. HTTP Basic: теория и реализация | 12. Шпаргалки и порядок действий |

1. Зачем веб-приложению безопасность

Веб-приложение получает запросы из недоверенной среды: браузера пользователя, мобильного клиента, curl, Postman или программы злоумышленника. Нельзя считать, что кнопка, скрытая в интерфейсе, защищает сервер: запрос к `post /units` можно отправить вручную. Защита должна жить на сервере.

Три базовые цели

Цель	Простой вопрос	Пример в проекте
Конфиденциальность	Кто не должен увидеть данные?	Пароль не возвращается из API; refresh в БД хранится только как hash.
Целостность	Кто не должен изменить данные?	Пользователь с <code>units_read</code> не может создать единицу.
Доступность	Кто не должен «уронить» сервис?	Ограничения запросов, защита от перебора паролей, разумные таймауты.

Типичные угрозы

Угроза	Что происходит	Базовая защита
Кража пароля	Фишинг, утечка, повторное использование одного пароля.	BCrypt для hash пароля, HTTPS, короткий access, 2FA, ограничение попыток.
Подбор пароля	Бот перебирает комбинации на логине.	Rate limit, временная блокировка, аудит, 2FA. Не выдавать слишком подробные ошибки.
Подмена запроса	Клиент меняет ID или шлёт закрытый endpoint.	@PreAuthorize, серверная проверка владельца/права, не доверять UI.
XSS	Чужой JavaScript выполняется на странице и читает доступные ему данные.	Не вставлять HTML пользователя как HTML; CSP; осторожно с localStorage.
CSRF	Чужой сайт заставляет браузер отправить запрос с его cookie.	CSRF-токен для cookie/session auth; для Bearer в заголовке риск другой.
Утечка токена	Кто-то получил bearer/access или refresh.	HTTPS, короткий access, refresh rotation, logout/revoke, не логировать секреты.
SQL injection	Ввод меняет смысл SQL.	JPA/параметризованные запросы, валидация; не собирать SQL конкатенацией.

HTTPS обязательно в реальном окружении. Basic, JWT и refresh без HTTPS можно перехватить в сети. JWT не заменяет TLS: он защищает целостность самого билета, но не шифрует весь HTTP-трафик.

2. Основные понятия

Термин	Человеческий смысл	В нашем проекте
Аутентификация (authentication)	Проверка «кто ты?».	Логин+пароль проверяются AuthenticationManager.
Авторизация (authorization)	Проверка «что тебе можно?».	@PreAuthorize("hasAnyAuthority(...)") на контроллере.
Principal	Представление текущего пользователя для Security.	AppPrincipal, содержащий User.
Authentication	Объект Security: principal + credentials + authorities + флаг authenticated.	После JWT-фильтра лежит в SecurityContext.
Authority	Одно конкретное разрешение.	UNITS_READ, USERS_WRITE, FULL_ACCESS.
Role	Группа разрешений; часто просто соглашение об именовании.	Сущность Role содержит набор Permission.
GrantedAuthority	Интерфейс Spring, которым представляется authority.	Permission.authority() даёт SimpleGrantedAuthority.
SecurityContext	«Карман» текущего HTTP-запроса, где Spring хранит Authentication.	JWT-фильтр кладёт туда пользователя до контроллера.
Session	Серверная память между запросами, обычно по cookie JSESSIONID.	У нас выключена: STATELESS.
CSRF	Подделка запроса через автоматически отправляемые cookie.	В stateless Bearer API CSRF выключен осознанно; если перейти на cookie, решение надо пересмотреть.

Role и authority: почему не одно и то же

Роль — удобное бизнесовое имя: «Кладовщик», «Администратор». Authority — атомарное действие: UNITS_READ. Можно дать двум разным ролям одно authority. В Spring выражение hasRole("ADMIN") обычно ищет authority ROLE_ADMIN; выражение hasAuthority("UNITS_READ") ищет строку буквально. Наш проект строится на конкретных permissions, поэтому hasAnyAuthority здесь правильнее, чем искусственно заменить всё на ADMIN.

3. Как работает Spring Security «из коробки»

Что такое HTTP, servlet и filter

Браузер отправляет HTTP-запрос: метод, URL, заголовки, тело. В Spring Boot встроенный веб-сервер (обычно Tomcat) принимает его. **Servlet** — Java-объект, который умеет обработать HTTP-запрос. Spring MVC использует центральный servlet `DispatcherServlet`: он находит нужный метод контроллера.

Filter — объект, который запускается *до* servlet и может либо пропустить запрос дальше, либо остановить/изменить его. Это удобно для безопасности: аутентификация должна произойти до вызова контроллера.



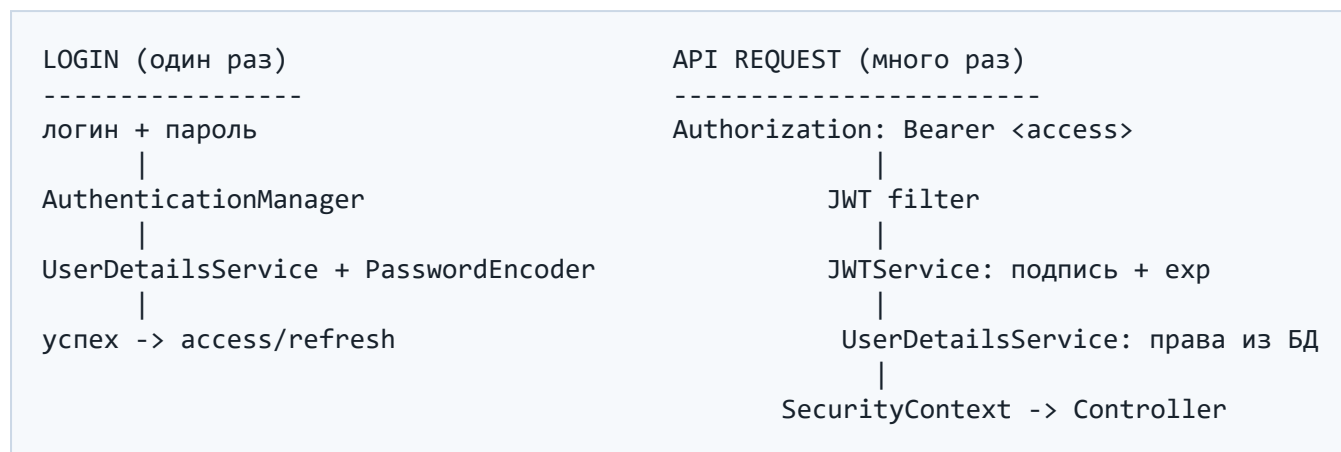
Что добавляет starter-security без настройки

Подключение `spring-boot-starter-security` добавляет фильтры безопасности. Если не написать собственную конфигурацию, Spring Boot обычно:

- закрывает почти все URL;
- создаёт временного пользователя и печатает пароль в лог;
- включает form login / Basic в зависимости от запроса;
- создаёт сессию после входа.

Это полезно как безопасный старт, но почти никогда не является готовой конфигурацией API. Мы заменяем поведение собственным `SecurityFilterChain`.

Два разных момента: login и обычный запрос



4. Виды аутентификации и 2FA

Факторы — это разные доказательства личности. Надёжная схема не обязана собирать все факторы; она выбирает их по риску и удобству.

Фактор	Что доказывает пользователь	Примеры	Слабые места
Knowledge-based («знаю»)	Секрет, который должен знать только он.	Пароль, PIN, контрольный вопрос.	Фишинг, повторное использование, утечки, подбор.
Possession-based («владею»)	Есть физический или цифровой предмет.	Телефон, аппаратный ключ FIDO2, карточка, приложение-аутентификатор.	Кража/потеря устройства, SIM-swap.
Inherence («являюсь»)	Биометрический признак.	Отпечаток, Face ID.	Ошибки распознавания; биометрию нельзя «сменить» как пароль.

Однофакторная и двухфакторная аутентификация

2FA — минимум два *разных* фактора. Пароль и второй пароль — не 2FA: оба «что знаю». Пароль + код из приложения — 2FA: «знаю» + «владею телефоном».

SMS-код

Сервер после пароля создаёт одноразовый код с коротким TTL, отправляет SMS и ждёт подтверждение. Плюс — пользователь уже умеет пользоваться SMS. Минусы — стоимость, доставка, перехват/переоформление SIM, фишинг. Для высокой безопасности SMS хуже TOTP/FIDO2, но лучше

одного пароля.

Google Authenticator и TOTP

Google Authenticator — один из клиентов протокола **TOTP** (Time-based One-Time Password). При включении 2FA сервер генерирует секрет и показывает QR-код. Приложение и сервер знают один секрет и одинаковое текущее время; оба считают короткий код, обычно меняющийся раз в 30 секунд. Сервер хранит секрет защищённо, не пароль от Google Authenticator.

Важно: Google Authenticator не «проверяет пользователя через Google». Это локальный генератор TOTP-кодов. Сервер сам проверяет код по общему секрету.

5. Базовая настройка Spring Security с нуля

Минимальная модель данных

User	---- many-to-many ----	Role	---- many-to-many ----	Permission
username, passwordHash		name		code = UNITS_READ
enabled				USERS_WRITE

Пароль никогда не хранится как обычный текст. В базе — только медленный salted hash BCrypt. При логине Spring вызывает `passwordEncoder.matches(rawPassword, passwordHash)`.

Порядок создания

1. Добавить `spring-boot-starter-security`.
2. Подготовить пользователя и `permissions` в БД.
3. Написать `AppPrincipal implements UserDetails`.
4. Написать `UserDetailsService`, который находит `principal` по `username`.
5. Объявить `PasswordEncoder (BCrypt)` и `AuthenticationManager`.
6. Написать `SecurityFilterChain`: публичные URL, защита остальных, `session policy`.
7. Поставить `@PreAuthorize` на действия, а не только скрыть кнопку на UI.

Фрагмент нашего SecurityConfiguration

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfiguration {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(12);
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity httpSecurity,
                                           JWTAuthenticationFilter jwtAuthenticationFilter) throws
Exception {
        return httpSecurity
            .csrf(AbstractHttpConfigurer::disable)
            .httpBasic(AbstractHttpConfigurer::disable)
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(a -> a
                .requestMatchers("/login/**", "/css/**", "/js/**", "/").permitAll()
                .anyRequest().authenticated())
            .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class)
            .build();
    }
}
```

Строка / тип	Зачем
@Configuration	Говорит Spring: этот класс создаёт бины конфигурации.
@EnableWebSecurity	Включает web security инфраструктуру.
@EnableMethodSecurity	Включает @PreAuthorize на методах.
@Bean PasswordEncoder	Один общий объект для hash и проверки пароля.
BCryptPasswordEncoder(12)	12 — work factor: безопасность выше, проверка медленнее. Это намеренно.
STATELESS	Не создавать/не читать HTTP session; каждый запрос несёт собственное доказательство.
requestMatchers(...).permitAll()	Публичные пути, которые можно вызвать до входа.
anyRequest().authenticated()	Без Authentication в SecurityContext дальше не пройти.
addFilterBefore	Наш JWT-фильтр запускается до стандартного username/password фильтра.

6. HTTP Basic

Теория

HTTP Basic передаёт в каждом запросе заголовок `Authorization: Basic Base64(username:password)`. Base64 — **кодировка, не шифрование**: строку легко декодировать. Basic допустим только через HTTPS и подходит для простых API, service-to-service или учебного старта. Для SPA хранить пароль и прикладывать его к каждому запросу — плохая долгосрочная схема.

Схема Basic



Как внедрить с нуля

1. Подключить Security starter.
2. Создать `UserDetails` и `UserDetailsService`.
3. Создать `BCrypt PasswordEncoder`.
4. В `SecurityFilterChain` включить `httpBasic(Customizer.withDefaults())`.
5. Для первого опыта отключить form login; выбрать session policy осознанно.
6. Поставить method security.
7. Проверить curl/Postman с Basic header.

Ключевые объекты нашего проекта

```
public class AppPrincipal implements UserDetails {
    private final User user;

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return user.getRoles().stream()
            .flatMap(role -> role.getPermissions().stream())
            .map(Permission::authority)
            .collect(Collectors.toSet());
    }

    @Override
    public String getPassword() { return user.getPasswordHash(); }

    @Override
    public String getUsername() { return user.getUsername(); }
}
```

Фрагмент	Пояснение
implements UserDetails	Адаптер между собственной сущностью User и контрактом Spring Security.
Collection<? extends GrantedAuthority>	Список разрешений. ? extends разрешает любой конкретный тип, реализующий интерфейс.
flatMap	У пользователя много ролей, у каждой много permissions. flatMap превращает «список списков» в один поток permissions.
map(Permission::authority)	Ссылка на метод: каждый Permission превращается в Spring-совместимый GrantedAuthority.
getPassword()	Возвращает hash из БД, не сырой пароль. Spring использует его ТОЛЬКО ДЛЯ matches.

```
@Service
@RequiredArgsConstructor
public class UserDetailsServiceImpl implements UserDetailsService {
    private final UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username) {
        User user = userRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException("..."));
        return new AppPrincipal(user);
    }
}
```

UserDetailsService отвечает только на вопрос «найди пользователя по логину и отдай Security-представление». Он **не** сравнивает пароль вручную: это задача AuthenticationManager и PasswordEncoder.

7. JWT (JSON Web Token)

Теория

JWT — компактный подписанный билет. Он обычно выглядит как три Base64URL-части: header.payload.signature. Это **не зашифрованный** контейнер: payload можно прочитать. Подпись доказывает, что header/payload выданы сервером и не изменены.

```
header:    { "alg": "HS256", "typ": "JWT" }
payload:   { "sub": "admin", "iat": ..., "exp": ... }
signature: HMAC-SHA256(base64url(header) + "." + base64url(payload), serverSecret)
```

У нас симметричная подпись HMAC: один secret и создаёт, и проверяет подпись. Он живёт на сервере в app.jwt.secret. При проверке JWT пересчитывает подпись тем же ключом и проверяет exp. Клиент ключ никогда не получает.

Схема JWT в нашем проекте

```
POST /login/auth { username, password }
-> AuthenticationManager -> UserDetailsService + BCrypt
-> JWTService.generateToken(username)
-> { accessToken: "eyJ..." }

GET /units/
Authorization: Bearer eyJ...
-> JWTAuthenticationFilter
-> JWTService.isTokenValid + extractUsername
-> UserDetailsService.loadUserByUsername
-> SecurityContextHolder.setAuthentication(...)
-> UnitController + @PreAuthorize
```

Алгоритм внедрения JWT

1. Добавить три JWT зависимости одной версии: api, impl, jackson.
2. Положить secret и срок access в конфиг (в проде — env/secret storage, не Git).
3. Создать JWTService: generate, parse, extract username, validate.
4. Создать filter, читающий Authorization: Bearer.
5. При валидном токене построить Authentication и положить в SecurityContext.
6. Создать контроллер логина через AuthenticationManager.
7. Отключить Basic, перейти на STATELESS, зарегистрировать фильтр.
8. Из фронта хранить access и прикладывать только к защищённым API.

JWTService: разбор текущего кода

```
public String generateToken(String username) {
    Instant now = Instant.now();
    return Jwts.builder()
        .subject(username)
        .issuedAt(Date.from(now))
        .expiration(Date.from(now.plus(tokenExpiration)))
        .signWith(signingKey())
        .compact();
}
```

Строка	Смысл
Instant now	Текущий момент по UTC; удобен для времени безопасности.
Jwts.builder()	Builder JWT: собирает claims и подпись пошагово.
subject(username)	Стандартный claim sub — идентификатор пользователя.
issuedAt	iat, время создания билета.
expiration	exp; после него билет недействителен.
signWith	Ставит криптографическую подпись ключом сервера.
compact()	Финально сериализует JWT в строку.

JWT filter: суть

```
String header = request.getHeader("Authorization");
if (header == null || !header.startsWith("Bearer ")) {
    filterChain.doFilter(request, response);
    return;
}
String token = header.substring("Bearer ".length()).trim();
if (SecurityContextHolder.getContext().getAuthentication() == null
    && jwtService.isValid(token)) {
    String username = jwtService.extractUsername(token);
    UserDetails user = userDetailsService.loadUserByUsername(username);
    var auth = new UsernamePasswordAuthenticationToken(
        user, null, user.getAuthorities());
    SecurityContextHolder.getContext().setAuthentication(auth);
}
filterChain.doFilter(request, response);
```

Нет Bearer — фильтр не является ошибкой сам по себе: публичный endpoint может работать. Но защищённый endpoint позднее увидит пустой SecurityContext и ответит 401. Три аргумента в UsernamePasswordAuthenticationToken означают «пользователь уже аутентифицирован»; второй аргумент null — пароль здесь не нужен и не должен жить в контексте.

8. JWT + refresh token

Зачем нужна пара

	Access	Refresh
Назначение	Ходить к API.	Получить новую пару без пароля.
Срок у нас	15 минут.	7 дней.
Вид	Подписанный JWT.	Случайный UUID.
Где проверяется	Подпись + срок в JWT filter.	Запись в PostgreSQL: hash, expiry, revoked.
Куда посылается	Каждый защищённый запрос.	Только /login/refresh.

Аналогия завода

Access — сменный пропуск на турникет, действующий 15 минут. Refresh — талон в бюро пропусков на неделю. Турникет не принимает талон; бюро не требует паспорт, пока талон жив. В журнале бюро фиксируется, какой талон кому выдали и не сдан ли он. Logout — сдать талон; текущий пропуск максимум 15 минут ещё может быть действителен, но новый уже не выдадут.

Полный алгоритм

1. Login: пароль -> access JWT + raw refresh UUID
2. БД: SHA-256(raw refresh), user, expiresAt, revoked=false
3. API: только Bearer access
4. Access expired: POST /login/refresh { refreshToken: raw }
5. БД: hash -> find -> !revoked -> !expired
6. Rotation: old.revoked=true; create new refresh; create new access
7. Клиент заменяет обе строки
8. Logout: revoke refresh (или все refresh пользователя) + очистить клиент

Почему SHA-256 для refresh и BCrypt для пароля

У пароля известен пользователь: можно взять его hash из записи и спросить BCrypt matches(raw, hash). У сырого refresh сначала нужно найти, *какая* строка ему соответствует. BCrypt от одной строки каждый раз выдаёт другой результат из-за соли, поэтому findByTokenHash(passwordEncoder.encode(raw)) не найдёт запись. SHA-256 детерминирован: один raw всегда даёт один hex hash, который можно индексировать и искать.

Состояние текущего проекта

Шаг	Статус	Факт
A. Конфиг	Готово	token_expiration: 15m, refresh_expiration: 7d.
B. Таблица/entity/repository	Готово	refresh_tokens, RefreshToken, findByTokenHash.
C. Выдача при login	Готово	AuthController выдаёт LoginResponse(accessToken, refreshToken).
D. Exchange + rotation	Надо сделать	POST /login/refresh, 401 на плохой token, revoke старого, новая пара.
E. Public URL	Уже покрыто	/login/** в permitAll.
F. Frontend retry	Надо сделать после D	JS пока ждёт data.token, а бэк возвращает accessToken.
Logout	Надо сделать	Отзыв refresh в БД и очистка токенов в браузере.

Как дописать D: refresh endpoint

1. Создать RefreshRequest(@NotBlank String refreshToken).
2. Создать exception, который в advice отдаёт 401 одинаково для not found/revoked/expired.
3. В сервисе пометить метод @Transactional.
4. Посчитать SHA-256 raw, найти строку по findByTokenHash.
5. Проверить revoked и expiresAt.
6. Поставить старому revoked=true.
7. Создать новый refresh через существующий createRefreshToken(username).
8. Создать access через JWTService.generateToken(username).
9. Вернуть LoginResponse.
10. Проверить Postman: login → refresh → повтор старого refresh = 401.

Redis для refresh: «круче» ли?

Нет, не «круче», а другой storage. Redis хорош для очень быстрых короткоживущих записей и умеет TTL. PostgreSQL лучше подходит текущему учебному проекту: уже есть Liquibase, запись переживает рестарт Redis, видно историю и revoked. Сначала закрыть протокол на Postgres. Redis имеет смысл позже при реальной нагрузке или отдельной учебной цели.

9. OAuth 2.0

Что это и чем не является

OAuth 2.0 — протокол **делегирования доступа**. Он отвечает на вопрос: «может ли приложение получить ограниченный доступ к ресурсу пользователя, не узнавая его пароль?» Сам OAuth не равен JWT, не равен login и не гарантирует, что вы знаете имя человека.

Пример: сайт-планировщик просит доступ к Google Calendar. Пароль Google вводится только на стороне Google. Планировщик получает access token с scope «читать календарь», но не пароль.

Участники

Роль	Смысл	Пример
Resource owner	Владелец данных.	Пользователь Google.
Client	Приложение, которое просит доступ.	Наш web-app.
Authorization server	Показывает login/consent и выдаёт tokens.	Google, Keycloak, GitHub.
Resource server	API с защищёнными данными.	Google Calendar API или наш API.
Scope	Граница разрешения для токена.	calendar.readonly.

Authorization Code + PKCE — современный поток для браузера

```
1. Client -> Authorization Server: /authorize?
client_id&redirect_uri&scope&code_challenge
2. User логинится у провайдера и подтверждает consent
3. Authorization Server -> redirect_uri?code=ONE_TIME_CODE
4. Client -> /token: code + code_verifier
5. Authorization Server -> access token (+ ID token при OIDC, возможно refresh)
6. Client -> Resource Server: Bearer access token
```

PKCE защищает код авторизации от перехвата. Клиент заранее создаёт случайный code_verifier, в /authorize отправляет только его производный code_challenge, а на /token раскрывает verifier. Украденный code без verifier обменять нельзя.

Другие grant type

Grant	Когда	Нюанс
Authorization code + PKCE	Пользователь входит в web/mobile.	Основной рекомендуемый поток.
Client credentials	Сервис вызывает сервис от своего имени.	Нет пользователя и его login.
Refresh token	Продлить делегированный доступ.	Не путать с самописным refresh, хотя идея похожа.
Device authorization	Устройство без удобной клавиатуры.	Телевизор, CLI, терминал.

Не используй Password Grant. Старый поток, где client получает пароль пользователя, противоречит главной идее OAuth и не рекомендуется для новых приложений.

10. OpenID Connect (OIDC)

Название «OpenID» в современном Java/Spring контексте почти всегда означает **OpenID Connect** (OIDC), слой идентификации поверх OAuth 2.0. OAuth говорит: «вот ограниченный доступ к API». OIDC добавляет: «вот проверяемое утверждение, кто пользователь».

ID token

OIDC выдаёт **ID token**, обычно JWT. Он адресован *клиентскому приложению*, а не ресурсному API. В нём встречаются claims `iss` (кто выдал), `aud` (для какого `client_id`), `sub` (стабильный ID пользователя), `exp`, `nonce`, иногда `name/email`.

Access token и ID token не взаимозаменяемы. API должен принимать access token, а frontend использует ID token/claims для сведений о вошедшем пользователе. Нельзя послать ID token в защищённый API «потому что это JWT».

Discovery и JWKS

Провайдер OIDC публикует `/.well-known/openid-configuration`: URL `/authorize`, `/token`, `issuer`, JWKS URI и поддерживаемые scopes. Через JWKS ресурсный сервер получает публичные ключи провайдера для проверки асимметричных подписей. Это отличается от нашего HMAC: у нас один секрет знают обе стороны; у OIDC private key остаётся у провайдера, а проверяющие получают public key.

11. Как реализовать OAuth 2.0 / OIDC в Spring

Сначала выбрать роль приложения

Наша роль	Spring starter	Когда
OAuth2 Client	spring-boot-starter-oauth2-client	«Войти через Google/Keycloak».
Resource Server	spring-boot-starter-oauth2-resource-server	Наш API принимает access tokens внешнего issuer.
Authorization Server	Spring Authorization Server или Keycloak	Мы сами выдаём OAuth tokens многим клиентам. Отдельная большая задача.

Вариант 1: Login через Keycloak / Google (OAuth2 Client + OIDC)

1. Зарегистрировать client у провайдера; получить client-id, redirect URI и (если confidential client) secret.
2. Добавить oauth2-client starter.
3. Добавить spring.security.oauth2.client.registration в yaml.
4. Разрешить /oauth2/** и /login/**.
5. Включить oauth2Login.
6. После успеха сопоставить внешний issuer + sub с внутренним User и его permissions.
7. Решить, как наш SPA получает сессию/собственный access; не смешивать случайно external ID token и API token.

```
dependencies {
    implementation("org.springframework.boot:spring-boot-starter-oauth2-client")
}

spring:
  security:
    oauth2:
      client:
        registration:
          keycloak:
            client-id: warehouse-web
            client-secret: ${KEYCLOAK_CLIENT_SECRET}
            scope: openid, profile
        provider:
          keycloak:
            issuer-uri: https://id.example.com/realms/warehouse
```

scope: openid переключает OAuth-клиента в OIDC-режим. issuer-uri позволяет Spring найти discovery-документ сам. Секреты не коммитятся в application.yaml.

Вариант 2: наш API — Resource Server

1. Добавить resource-server starter.
2. Настроить issuer-uri провайдера.
3. Заменить самописный JWT filter на `oauth2ResourceServer(o -> o.jwt())` после отдельного изучения migration.
4. Настроить converter claims/scopes внешнего token в наши GrantedAuthority.
5. Оставить бизнес-проверки на уровне метода/сервиса.

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://id.example.com/realms/warehouse

http.oauth2ResourceServer(oauth2 -> oauth2.jwt());
```

Что выбрать этому проекту сейчас

Ничего из OAuth/OIDC пока не внедрять поверх незавершённого refresh. У нас один SPA, один backend, свои пользователи и permissions. Самописный login+JWT — учебно оправдан. OAuth/OIDC стоит брать отдельной темой после security-конспекта: например, Keycloak как внешний IdP и сценарий SSO. Распиливать auth в микросервис или поднимать Authorization Server «для галочки» не нужно.

12. Шпаргалки

Что сказать себе утром перед работой

Формула: Login проверяет пароль и выдаёт билет. Обычный запрос проверяет билет и наполняет SecurityContext. Авторизация смотрит authorities. Refresh не ходит к API: он меняет старый длинный талон на новую пару билетов.

Чеклист: добавить защищённую сущность

1. Добавить коды permissions в enum и данные миграцией.
2. Убедиться, что `AppPrincipal.getAuthorities()` получает эти permissions через роли.
3. Поставить `@PreAuthorize` на read/write методы.
4. Проверить 401 без access, 403/доступ с разными authorities.
5. Доработать фронтенд-права по правилу проекта; UI — удобство, сервер — реальная защита.

Чеклист: закончить refresh

1. DTO RefreshRequest.
2. Единая 401 ошибка для invalid/revoked/expired refresh.
3. Transactional validate + revoke old + create new.
4. POST /login/refresh.
5. Postman rotation test.
6. Frontend: accessToken и refreshToken; один retry на 401.
7. Logout revoke + очистка storage.
8. Тесты auth/refresh.

Никогда не делать

- Не класть сырой пароль, JWT secret, client secret или сырой refresh в Git, логи и exception message.
- Не считать Base64 шифрованием.
- Не доверять скрытой кнопке как проверке права.
- Не использовать ID token вместо access token в API.
- Не отключать CSRF «на всякий случай» при переходе на cookie/session auth.
- Не превращать OAuth в задачу «сделать JWT посложнее»: это разные уровни.

Конец конспекта. Актуализируй разделы 8 и 12 после завершения refresh/logout и перед внедрением OAuth/OIDC.